JAVA COURSE

{ metodi }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

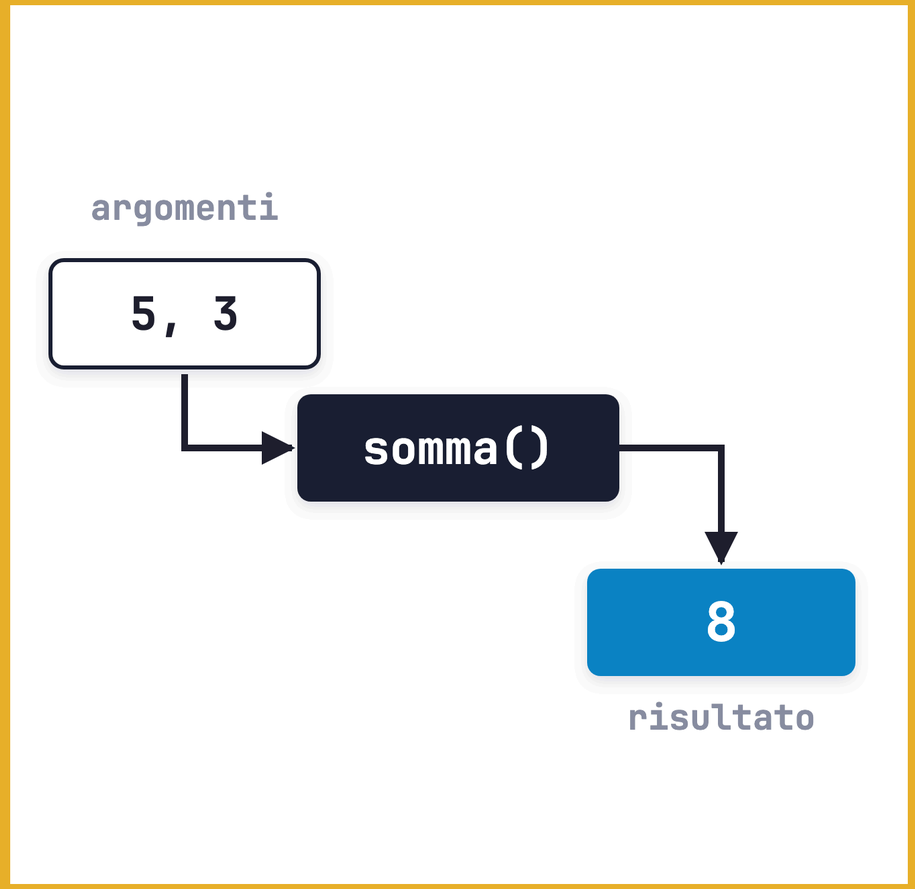
01 Cos'è un metodo

02 Parametri e ritorno

03 Metodi in pratica

04 Da main ai metodi

01
COS'È UN METODO



PERCHÉ SERVONO I METODI

Finora tutto il codice stava nel `main`: funzionava, ma blocchi identici comparivano più volte e il programma cresceva senza ordine

- ***Dare un nome*** *a un blocco di istruzioni*
- ***Riusarlo*** *ogni volta che serve, senza riscriverlo*
- ***Scomporre il problema*** *in parti piccole e leggibili*

COS'È UN METODO

Un **metodo** è un blocco di codice con un nome, dichiarato dentro una classe. Si scrive una volta e si invoca quante volte si vuole

```
static void stampaSaluto() {
    System.out.println("Ciao, studente!");
}

public static void main(String[] args) {
    stampaSaluto();   // Ciao, studente!
    stampaSaluto();   // Ciao, studente!
}
```

ANATOMIA DI UN METODO

Ogni metodo ha cinque parti

- ***Modificatori:*** `static`, *accesso*
- ***Tipo di ritorno:*** `void` *o un tipo*
- ***Nome:*** *camelCase, un verbo*
- ***Parametri:*** *tra parentesi tonde*
- ***Corpo:*** *tra parentesi graffe*

```
static void stampaSaluto() {
//mod. tipo nome      param.
    System.out.println("Ciao!");
//          corpo
}
```

IL PRIMO METODO

Un metodo `void` senza parametri esegue sempre le stesse istruzioni

```java
static void stampaLinea() {
    System.out.println("----------");
}

public static void main(String[] args) {
    System.out.println("Sezione 1");
    stampaLinea();
    System.out.println("Sezione 2");
    stampaLinea();
}
```

Terminale

```
Sezione 1
----------
Sezione 2
----------
```

IL FLUSSO DI ESECUZIONE

L'esecuzione non scorre dall'alto in basso nel file. L'ordine lo decide il codice, non la posizione

- *Parte dal* `main`
- *Quando incontra un'invocazione,* ***salta al metodo***
- *Esegue il corpo del metodo fino in fondo*
- ***Torna al punto di invocazione*** *e prosegue*
- *È un'esecuzione* ***sincrona*** *o* ***asincrona****?*

METODI GIÀ USATI

`System.out.println`, `Math.pow`, `Math.random` sono tutti metodi della libreria standard

- *La differenza: quelli li ha scritti qualcun altro, ora impariamo a scriverne di nostri*

```
double potenza = Math.pow(2, 10);        // metodo con ritorno
double r = Math.random();                // metodo con ritorno
System.out.println("Hello");             // metodo void
```

CHE COSA STAMPA?

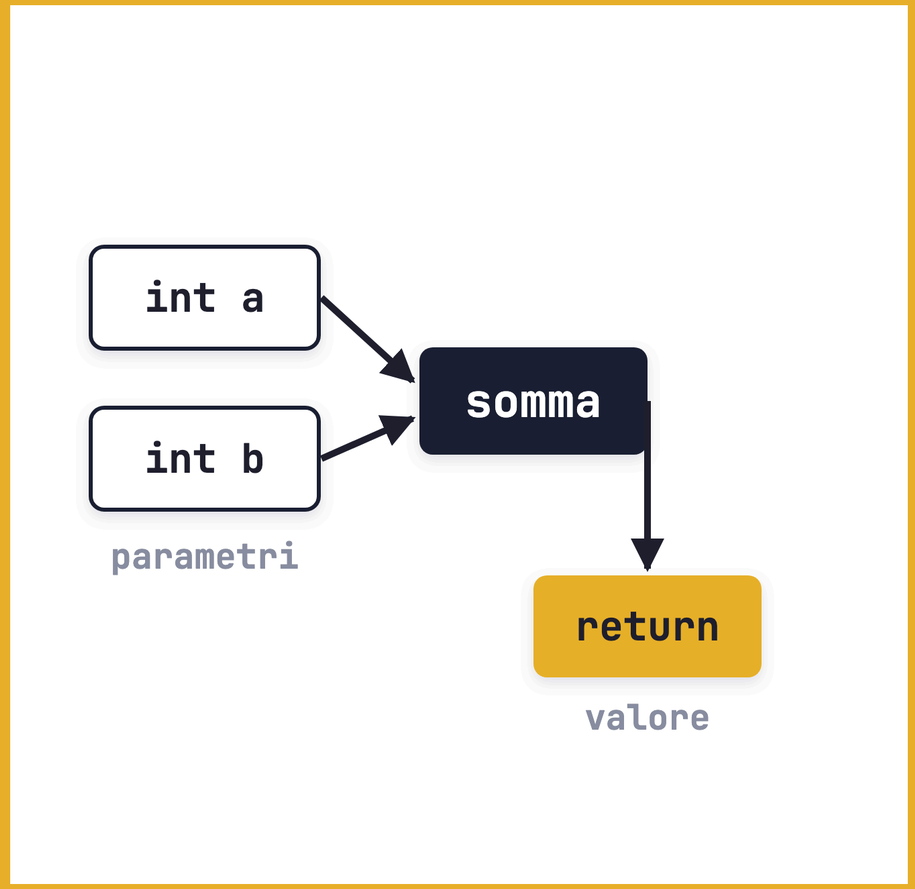
Che cosa stampa questo programma?

```
public class Quiz {

    static void messaggio() {
        System.out.println("Dentro il metodo");
    }

    public static void main(String[] args) {
        System.out.println("Prima invocazione");
        messaggio();
        System.out.println("Tra le due invocazioni");
        messaggio();
        System.out.println("Dopo la seconda invocazione");
    }
}
// ??
```

02
PARAMETRI E RITORNO



I PARAMETRI

Un metodo può ricevere dati dall'esterno attraverso i **parametri**: variabili dichiarate tra le parentesi tonde, usabili nel corpo del metodo

```
static void saluta(String nome) {
    System.out.println("Ciao, " + nome + "!");
}

public static void main(String[] args) {
    saluta("Alice");   // Ciao, Alice!
    saluta("Bob");     // Ciao, Bob!
}
```

PARAMETRI E ARGOMENTI

Il **parametro** è nella dichiarazione del metodo, l'**argomento** è il valore passato nell'invocazione. Sono la stessa cosa, ma in momenti diversi

```
static void saluta(String nome) {   // ← parametro
    System.out.println("Ciao, " + nome + "!");
}

saluta("Alice");   // ← argomento
```

PIÙ PARAMETRI

Si possono dichiarare più parametri separati da virgola. Ogni parametro dichiara il proprio tipo, anche se è uguale al precedente

```
static void stampaOrario(int ore, int minuti) {
    System.out.printf("%02d:%02d%n", ore, minuti);
}

public static void main(String[] args) {
    stampaOrario(9, 5);    // 09:05
    stampaOrario(14, 30);  // 14:30
}
```

IL PASSAGGIO PER VALORE

Java **copia** il valore dell'argomento nel parametro. Modificare il parametro dentro il metodo non cambia la variabile originale

```
static void raddoppia(int n) {
    n = n * 2;
    System.out.println("Dentro: " + n);   // 20
}

public static void main(String[] args) {
    int x = 10;
    raddoppia(x);
    System.out.println("Fuori: " + x);    // 10 — invariato
}
```

VOID

`void` significa che il metodo non restituisce nessun valore. Il suo scopo è produrre un **effetto**: stampare, modificare uno stato, aprire un file

- `return;` *senza valore è opzionale nei metodi* `void`*: l'esecuzione torna al chiamante anche senza*

```
static void stampaPositivo(int n) {
    if (n <= 0) {
        return;            // uscita anticipata
    }
    System.out.println(n);
}
```

IL RETURN

Un metodo con un tipo di ritorno diverso da `void` deve restituire un valore con `return`. Il tipo del valore deve corrispondere al tipo dichiarato

```
static int somma(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int risultato = somma(3, 4);
    System.out.println(risultato);   // 7
}
```

USARE IL VALORE RESTITUITO

Il valore restituito da un metodo si può assegnare a una variabile, stampare direttamente o usare dentro un'espressione

```
static int quadrato(int n) {
    return n * n;
}

public static void main(String[] args) {
    int q = quadrato(5);                        // assegnato
    System.out.println(quadrato(3));            // stampato: 9
    System.out.println(quadrato(2) + quadrato(3));  // in espressione: 13
}
```

MISSING RETURN STATEMENT

Se esiste un percorso di esecuzione che non raggiunge un `return`, il compilatore si ferma. Non è un errore a runtime: `javac` lo individua prima

```
static String classifica(int punti) {
    if (punti >= 60) {
        return "promosso";
    }
    // errore: missing return statement
    // se punti < 60, non c'è return
}
```

IL CODICE MORTO

Le istruzioni scritte dopo un `return` non vengono mai eseguite: si chiamano **codice irraggiungibile**. `javac` segnala l'errore

```
static int valore() {
    return 42;
    System.out.println("Mai");   // unreachable statement
}
```

RETURN COME USCITA ANTICIPATA

In un metodo `void`, `return;` esce subito senza eseguire il resto del corpo.

Usato all'inizio per controllare i casi non validi si chiama **guard clause**

```
static void stampaRadice(double n) {
    if (n < 0) {
        System.out.println("Numero negativo");
        return;                          // uscita anticipata
    }
    System.out.println(Math.sqrt(n));
}

public static void main(String[] args) {
    stampaRadice(-4);   // Numero negativo
    stampaRadice(9);    // 3.0
}
```

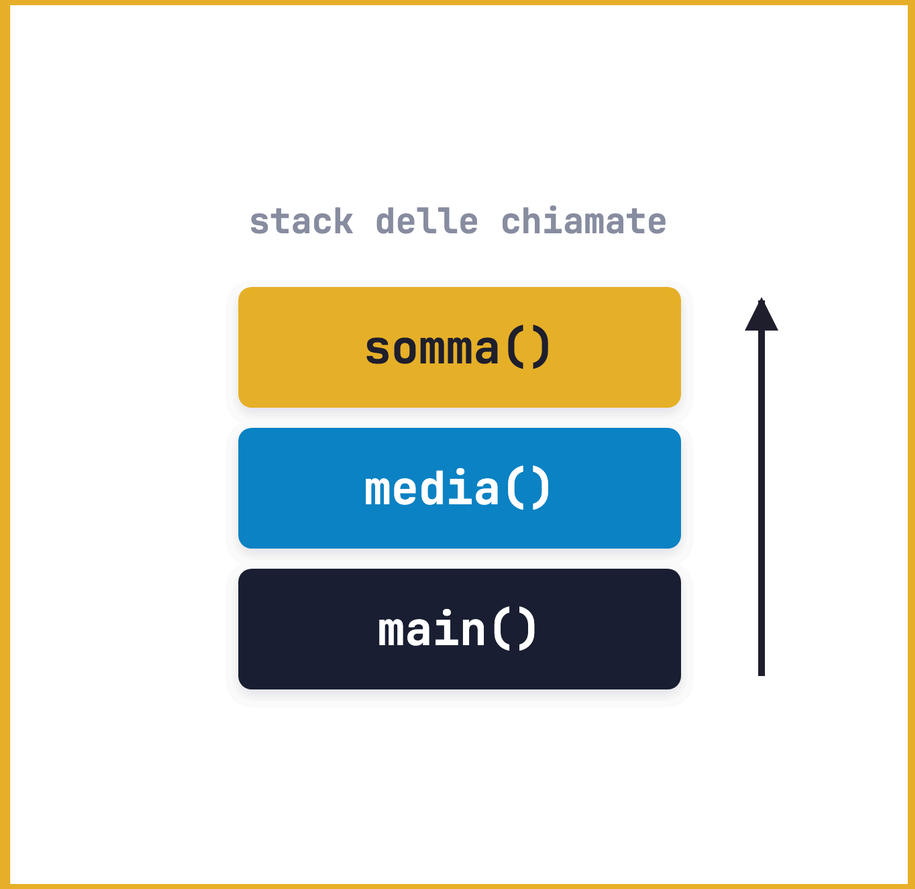
CHE COSA RESTITUISCE?

Qual è il valore restituito da `calcola(3, 7)`?

```
static int calcola(int a, int b) {
    if (a > b) {
        return a - b;
    }
    return a + b;
}

int r = calcola(3, 7);
System.out.println(r);
// ??
```

03
METODI IN PRATICA



LA FIRMA

La **firma** di un metodo identifica univocamente quel metodo all'interno della classe. È composta dal nome e dall'elenco dei tipi dei parametri

- *Il tipo di ritorno **non fa parte** della firma*
- `somma(int, int)` *è la firma —* `int somma(int a, int b)` *è la dichiarazione completa*

L'OVERLOADING

Nella stessa classe possono esistere più metodi con lo stesso nome, purché abbiano **firme diverse**. Si chiama **overloading**

- *Lo conosciamo già:* `System.out.println` *accetta* `int`, `String`, `double` *e molti altri tipi — sono metodi con la stessa origine ma firme diverse*

```
System.out.println(42);        // println(int)
System.out.println("Ciao");    // println(String)
System.out.println(3.14);      // println(double)
```

OVERLOADING: ESEMPIO

Due metodi con lo stesso nome `somma` ma parametri di tipo diverso: il compilatore sceglie quello giusto in base agli argomenti

```
static int somma(int a, int b) {
    return a + b;
}

static double somma(double a, double b) {
    return a + b;
}

public static void main(String[] args) {
    System.out.println(somma(3, 4));         // 7 — chiama somma(int, int)
    System.out.println(somma(1.5, 2.5));     // 4.0 — chiama somma(double, double)
}
```

LE VARIABILI LOCALI

I parametri e le variabili dichiarate dentro un metodo esistono solo all'interno di quel metodo: sono **variabili locali.** Due metodi non condividono le loro variabili locali

```
static void metodoA() {
    int valore = 10;
    System.out.println(valore);
}

static void metodoB() {
    System.out.println(valore);   // errore: cannot find symbol
}
```

LO STACK DELLE INVOCAZIONI

Ogni invocazione di un metodo crea un **frame** sullo stack con le proprie variabili locali. Quando il metodo termina, il frame viene rimosso e le variabili spariscono

- ***main*** *invoca* `somma` *→ si crea un frame con* `a`, `b`
- `somma` *termina con* `return` *→ il frame viene tolto dallo stack*
- *Il controllo torna al frame di* ***main***

METODI CHE CHIAMANO METODI

Un metodo può invocare un altro metodo: lo stack cresce di un frame per ogni invocazione in sospeso

```
static int quadrato(int n) {
    return n * n;
}

static int sommaDeiQuadrati(int a, int b) {
    return quadrato(a) + quadrato(b);   // invoca quadrato due volte
}

public static void main(String[] args) {
    System.out.println(sommaDeiQuadrati(3, 4));   // 25
}
```

LA RICORSIONE

Un metodo può invocare **se stesso**: si chiama **ricorsione**. Ogni invocazione ricorsiva aggiunge un frame allo stack

- *Serve sempre un **caso base** che fermi la catena: senza di esso lo stack si riempie e il programma termina con* `StackOverflowError`

```
static int fattoriale(int n) {
    if (n == 0) {
        return 1;                // caso base
    }
    return n * fattoriale(n - 1);   // chiamata ricorsiva
}
// fattoriale(4) → 4 * 3 * 2 * 1 = 24
```

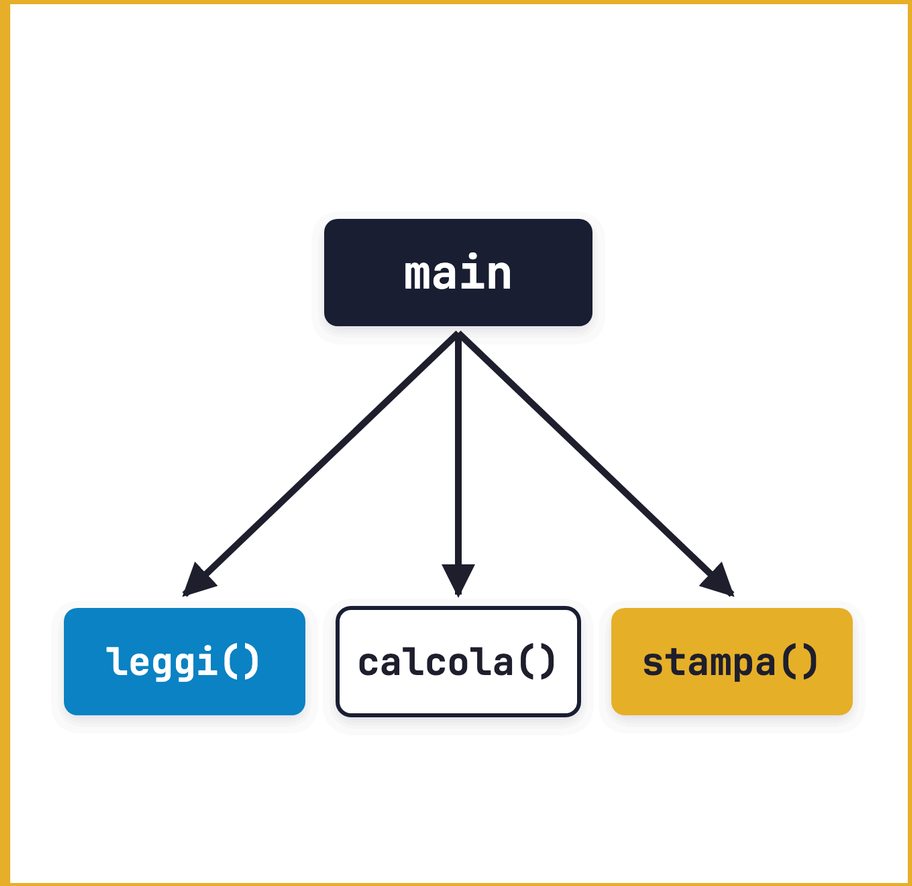
QUANDO CREARE UN METODO

Non esiste una regola rigida, ma ci sono segnali che indicano che è il momento di estrarre un metodo

Situazione	Segnale
Codice ripetuto in più punti	estrarre un metodo
Blocco di codice troppo lungo	suddividere in metodi
Un nome renderebbe più chiaro	estrarre un metodo
Un calcolo serve in più punti	metodo con `return`

04

DA MAIN AI METODI



REFACTORING: ESTRARRE UN METODO

Il codice ripetuto nel `main` diventa un metodo: il comportamento non cambia, ma il codice è più corto e leggibile. Si chiama **refactoring**

```java
// Prima
public static void main(...) {
  System.out.println("---");
  System.out.println("A");
  System.out.println("---");
  System.out.println("B");
  System.out.println("---");
}
```

```java
// Dopo
static void separatore() {
    System.out.println("---");
}

public static void main(...) {
    separatore();
    System.out.println("A");
    separatore();
    System.out.println("B");
    separatore();
}
```

STATIC

La parola chiave `static` appare prima del tipo di ritorno. Per ora tutti i metodi la portano perché li invochiamo senza creare un oggetto

- *Quando studieremo le classi e gli oggetti,* `static` *avrà un significato preciso: appartiene alla classe, non a un'istanza specifica*

LE CONVENZIONI

I nomi dei metodi seguono regole condivise da tutto l'ecosistema Java

- *Si usa **camelCase**: prima lettera minuscola, iniziali delle parole successive maiuscole*
- *Il nome è un **verbo**:* `calcola`, `stampa`, `converti`, `leggi`
- *I metodi che restituiscono* `boolean` *iniziano con* `is` *o* `has`*:* `isPari`, `hasValore`
- *Nomi precisi:* `calcolaMedia` *è meglio di* `calcola`

JAVA 21 E JAVA 25

In Java 21 i metodi vivono dentro una classe pubblica con `public static void main`. In Java 25 si possono scrivere in una **classe implicita**, senza `public` né `static`

Aspetto	Java 21	Java 25
Dichiarazione classe	`public class`	implicita
Modificatori su `main`	`public static`	nessuno
Metodi ausiliari	`static` **obbligatorio**	`static` **facoltativo**
Classe implicita	preview (JEP 445)	definitiva (JEP 512)

LE DUE FORME A CONFRONTO

Lo stesso programma scritto nelle due forme: il risultato è identico

```java
// Java 21
public class Saluti {

    static String formula(String nome) {
        return "Ciao, " + nome + "!";
    }

    public static void main(String[] args) {
        System.out.println(formula("Alice"));
    }
}
```

```java
// Java 25 (classe implicita)

String formula(String nome) {
    return "Ciao, " + nome + "!";
}

void main() {
    System.out.println(formula("Alice"));
}
```

CONFRONTO CON PYTHON

Chi arriva da Python trova differenze importanti sui metodi

Aspetto	Python	Java
Dichiarazione	`def`, nessun tipo	tipo di ritorno obbligatorio
Ritorno	`None` implicito	`void` esplicito
Tipi dei parametri	non dichiarati	dichiarati e controllati
Parametri di default	`def f(x=10)`	non esistono: si usa l'overloading
Scope	la variabile resta dopo il ciclo	la variabile muore col blocco
Overloading	non esiste	stesso nome, firme diverse

ERRORI COMUNI

I messaggi reali di `javac`

Errore	Messaggio di javac
Il metodo non restituisce in tutti i percorsi	`missing return statement`
Codice dopo un `return`	`unreachable statement`
Metodo `void` che prova a restituire un valore	`incompatible types: unexpected return value`
Tipo di ritorno sbagliato	`incompatible types`
Variabile di un altro metodo	`cannot find symbol`
Stessa firma in due metodi	`method is already defined in class`
Numero o tipo di argomenti sbagliato	`method cannot be applied to given types`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1